

WEEK 04 · LAYOUT II

THE PAGE GETS A SECOND DIMENSION

CSS Grid, and a page that fits every screen on purpose.

WHERE WE ARE

You can put two things side by side. You cannot yet make a page.

AFTER TODAY YOU CAN

Lay out a whole page in two directions, and re-lay it at any width.

WHAT THIS UNLOCKS

Every page you build from here. And next week Tailwind just renames it.

THIS IS WHAT YOU BUILT LAST WEEK

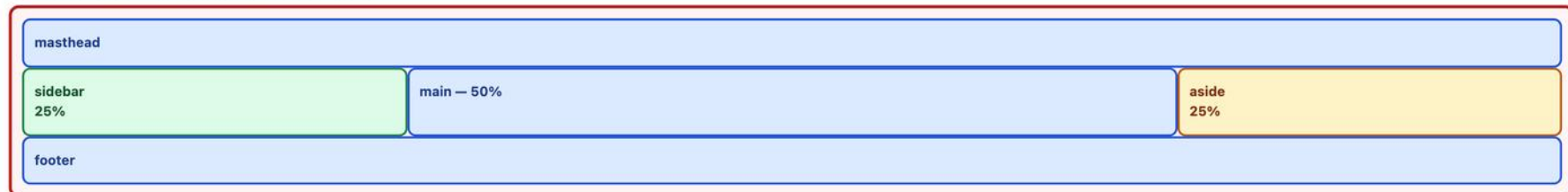
the page shell, attempted in honest Flexbox

↻ reload

open ↗

☐ add a gap between the regions ☐ shrink the percentages to compensate ☐ give the main region more content

FIVE REGIONS, ONE PAGE SHELL, FLEXBOX ONLY



The sidebar stops 48px above the footer.

```
container inner width  1498px
the band wants        1498px
gutter, sidebar to main 0px
sidebar bottom to footer 48px
wrapper elements needed 2
```

Nothing is wrong yet, and you should say so. Three percentages that add up to 100 do lay out three columns, and week 3 got you here honestly.

► **HANDS-ON** [demos/01-why-grid/01-flex-attempt/](#) — tick **ADD A GAP**, then **MORE CONTENT**. Do not defend it.

FLEXBOX REASONS ABOUT ONE AXIS AT A TIME

WHAT YOU ASKED FOR

A sidebar, a main column and an aside that line up as a **row** — and a masthead and footer that line up with them as a **column**.

WHAT FLEXBOX HEARD

A row. Then, separately, another row. Nothing connects the two, because a flex container has no concept of the column its items share with anything outside it.

WHAT GRID IS

A layout system that holds **both** directions at once, in the container, before any item has an opinion.

ACT 1 · TRACKS AND LINES

EIGHT LINES OF CSS, AND NO WRAPPER ELEMENTS

the same five regions, in Grid

↺ reload

open ↗

Show me the CSS that does this

THE SAME FIVE REGIONS, AND NO WRAPPER ELEMENTS

masthead

sidebar

main

The main region holds more than the others, exactly as before. Nothing about that is a problem any more.

aside

footer

Both gutters are 12px. The sidebar meets the footer exactly.

```
container inner width  1498px
the grid wants        1498px
gutter, sidebar to main 12px
sidebar bottom to footer 12px
wrapper elements needed 0
the browser resolved 1fr 2fr 1fr to
                        368.5px 737px 368.5px
```

► **HANDS-ON** [demos/01-why-grid/02-grid/](#) — ask them to guess the line count **BEFORE** you press the button.

YOU DEFINE THE CONTAINER FIRST

what it is A two-dimensional layout system, declared on the **container**.

key trait The tracks exist before any item does. Items then land in them, or are told where to go.

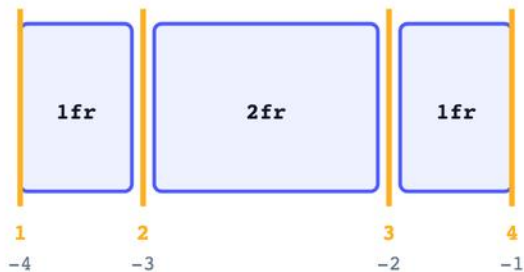
use cases Page shells. Card rows. Anything with rows *and* columns.

tip `display: grid` alone gives you **one column**, which looks exactly like nothing happened.

```
1 .page {  
2   display: grid;  
3   grid-template-columns: 1fr 2fr 1fr;  
4   grid-template-rows: auto 1fr auto;  
5   gap: 16px;  
6 }  
7  
8 .masthead {  
9   grid-column: 1 / -1;  
10 }
```

ITEMS SIT BETWEEN LINES, NOT IN CELLS

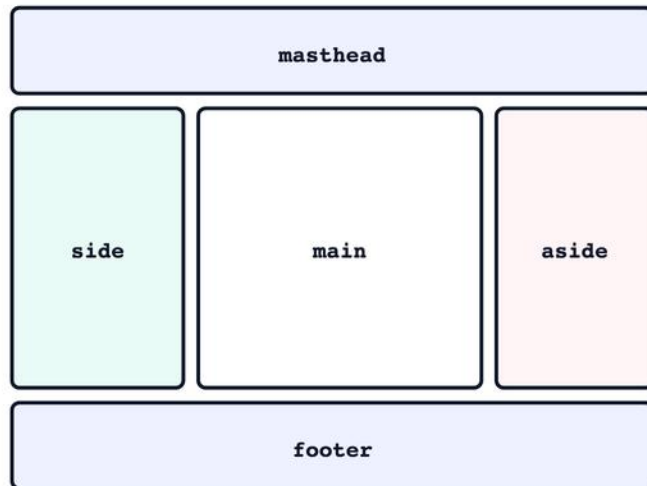
tracks, and the lines that come with them



an item sits between two lines



or draw the layout, and let the items claim a name



```
"masthead masthead masthead"  
"side    main    aside  "  
"footer  footer  footer  "
```

Three tracks. Four lines. And the same line counted backwards.

THE TOOL YOU WILL USE ALL EVENING

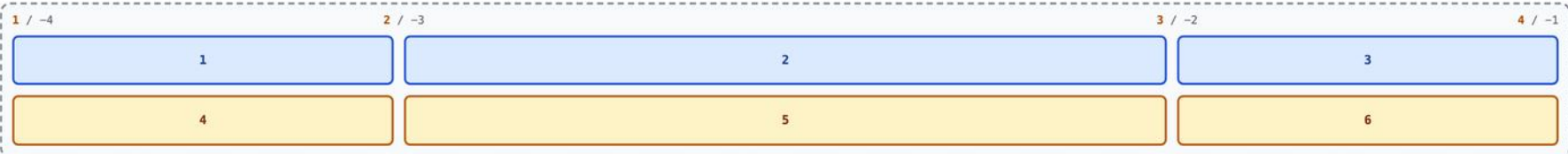
demos/02-tracks — type a track list, watch the browser answer reload open

200px 200px 200px 33% 33% 33% 1fr 1fr 1fr **1fr 2fr 1fr** auto 400px 1fr repeat(3, 1fr) repeat(4, minmax(120px, 1fr)) min-content max-content 1fr

grid-template-columns 1fr 2fr 1fr

gap 10px items 6

THE TRACKS, AND THE LINES THAT COME WITH THEM



3 tracks, so 4 lines, numbered 1 to 4.

```
you wrote 1fr 2fr 1fr
browser used 370px 740px 370px
gap 10px
items 6 in 2 rows
```

Read the second line, not the first. That is what the layout actually resolved to, in pixels — the USED value. Every track you wrote as a

► **HANDS-ON** demos/02-tracks/ — start on 200px 200px 200px and resize the window.

WHEN A TRACK LIST IS WRONG, NOTHING TURNS RED

A STRAY COMMA

```
1 .row {  
2   grid-template-columns: 1fr, 1fr, 1fr;  
3 }
```

Dropped. Falls back to `none`. You get one column.

THE FIX

```
1 .row {  
2   grid-template-columns: 1fr 1fr 1fr;  
3 }
```

Track lists are space-separated. Commas live inside `repeat()`.

► **THE POINT** — no console error, no red, no 404. It just looks like Grid did nothing.

ACT 2 · SIZING

fr IS NOT PERCENT WITH A NICER NAME

same intent, two units, one gap slider

↻ reload

open ↗

gap  0px ☐ trim the percentages to 32%

grid-template-columns: 33.333% 33.333% 33.333%



grid-template-columns: 1fr 1fr 1fr



container inner width 1498px

gap x 2 0px

percent resolved to 499.328px 499.328px 499.328px
and the row then wants 1498px (0)

fr resolved to 499.328px 499.328px 499.344px

With no gap these are the same layout, and you should say so plainly.
Nothing so far distinguishes the two units.

Now drag the gap. Every real design has one.

► **HANDS-ON** [demos/03-fr-vs-percent/](#) — gap at 0, then drag it in one motion and stop talking.

THE SIZING VOCABULARY

200px

Fixed. Correct when the thing genuinely has a size.

1fr

A share of the space that is **left over**.

auto

As big as the content, then it will stretch if there is spare room.

min-content

The narrowest it can be without overflowing.

max-content

As wide as it would like, wrapping nothing.

minmax(a, b)

A floor and a ceiling. The most useful function in this lesson.

```
1 .a {  
2   grid-template-columns:  
3     200px 1fr 200px;  
4 }  
5  
6 .b {  
7   grid-template-columns:  
8     repeat(4, 1fr);  
9 }  
10  
11 .c {  
12   grid-template-columns:  
13     repeat(3, minmax(10rem, 1fr));  
14 }
```

`repeat()` takes a **pattern**, not just one track.

YOU HAVE MET THIS BUG. IT IS THE SAME BUG.

why 1fr overflows, and what minmax(0, 1fr) fixes

[↶ reload](#)[open ↷](#)

☐ paste a long URL into the content ☐ try overflow: hidden ☐ try a narrower sidebar

grid-template-columns: 200px 1fr

sidebar

Ordinary text wraps, so nothing is wrong until something cannot wrap.

grid-template-columns: 200px minmax(0, 1fr)

sidebar

Ordinary text wraps, so nothing is wrong until something cannot wrap.

```
container inner width    1498px
200px 1fr               wants 1498px (0)
200px minmax(0, 1fr)    wants 1498px
resolved, top grid       200px 1288px
resolved, bottom grid    200px 1288px
```

Nothing is wrong yet, and nothing will be while the content can wrap.
That is why this bug reaches production: it needs a URL, a long token,
a wide <pre> or a table, and none of those are in anybody's test data.

Tick the first box.

► **HANDS-ON** [demos/07-minmax-0/](#) — take the two wrong fixes seriously before you give the right one.

1fr IS minmax(auto, 1fr)

BURSTS ON A LONG URL

```
1 .layout {  
2   grid-template-columns: 200px 1fr;  
3 }
```

THE FLOOR IS LOWERED

```
1 .layout {  
2   grid-template-columns: 200px minmax(0, 1fr);  
3 }
```

► **SAY IT OUT LOUD** — this course's own slide theme puts `min-width: 0` on every `.cols` child for exactly this reason.



ACT 3 · THE ONE EVERYBODY GETS WRONG

A CARD ROW THAT NEEDS NO MEDIA QUERY

```
1 .cards {  
2   display: grid;  
3   grid-template-columns: repeat(auto-fit, minmax(15rem, 1fr));  
4   gap: 1rem;  
5 }
```

As many columns as fit, never narrower than 15rem, sharing what is left.

Four cards or nine, phone or monitor, and not one breakpoint.

AUTO-FIT AND AUTO-FILL, SIDE BY SIDE

demos/04-auto-fit-fill — two sliders, one difference ↻ reload open ↗

container width  700px items  8

```
repeat(auto-fill, minmax(140px, 1fr))
```



```
repeat(auto-fit, minmax(140px, 1fr))
```



Right now these two are identical, and that is the normal case.

container	700px
items	8

► **HANDS-ON** demos/04-auto-fit-fill/ — start at 8 items, where they are **IDENTICAL**. Then drag the count down.

THE DIFFERENCE, AS A DECISION

auto-fit

Empty tracks collapse to zero. The items you have fill the row.

Use for a feature row, a card grid, a gallery.

auto-fill

Empty tracks stay. The rhythm of the columns is preserved.

Use for a calendar, a seating chart, a shelf.

THE TRAP

They are **identical** whenever there are enough items. Which is why almost every tutorial uses them interchangeably.

► **THE RULE** — a missing item in a calendar is information. A missing card is not.

ACT 4 · PLACEMENT

PUTTING AN ITEM SOMEWHERE ON PURPOSE

demos/05-placement/01-lines

↻ reload

open ↗

column-start 1 column-end 3 row-start 1 row-end 2 column-end = -1 toggle 4 / 6 columns

A 4 X 3 GRID. ONE ITEM IS PLACED; THE REST FLOW AROUND IT.



```
.placed {  
  grid-column: 1 / 3;  
  grid-row: 1 / 2;  
}
```

```
/* the same placement, written with span */
```

```
.placed {  
  grid-column: 1 / span 2;
```

► **HANDS-ON** demos/05-placement/01-lines/ — set 1 / 2 and ask why it covers one column.

FOUR WAYS TO SAY THE SAME PLACEMENT

```
1  .a {  
2    grid-column: 1 / 3;  
3    grid-row: 2 / 4;  
4  }  
5  
6  .b {  
7    grid-column: 1 / span 2;  
8    grid-row: 2 / span 2;  
9  }  
10  
11 .c {  
12   /* row-start / column-start / row-end / column-end */  
13   grid-area: 2 / 1 / 4 / 3;  
14 }  
15  
16 .d {  
17   grid-column: 1 / -1;  
18   /* all the way across, whatever the track count */  
19 }
```

OR DRAW THE LAYOUT AND LET THE ITEMS CLAIM A NAME

demos/05-placement/02-areas – the picture is the layout

reloadopen

desktoptabletphonea gap in the gridnot a rectangleagged rows

grid-template-areas

```
"masthead masthead masthead masthead"
"sidebar main main main aside"
"sidebar main main main aside"
"footer footer footer footer"
```

THE LAYOUT THAT PICTURE PRODUCES

masthead

sidebarmainaside

footer

Valid. Every row has the same number of cells and every named area is a rectangle.

A dot is a deliberately empty cell – useful, and the reason "a gap in the grid" is a preset here rather than a mistake.

Note what the five regions themselves say: ``grid-area: main`` and nothing else. The picture holds the layout; the items only claim a name. That is what makes a responsive page shell three pictures in three media queries

► **HANDS-ON** demos/05-placement/02-areas/ — edit a row, then press **PHONE**, then press **NOT A RECTANGLE**.

TWO RULES AREAS WILL NOT FORGIVE, AND WILL NOT MENTION

RAGGED ROWS

```
"head head head"  
"side main main aside"  
"foot foot foot"
```

Every row needs the same number of cells.

NOT A RECTANGLE

```
"side main main aside"  
"side main aside aside"
```

A named area must be a rectangle.
There is no L shape.

WHAT HAPPENS

The declaration is dropped, the property falls back to `none`, and you get plain auto-placement.

Nothing is logged.

THE GRID YOU DID NOT DECLARE

implicit grid Items past the end of your explicit tracks generate new rows. Sized by content unless you say otherwise.

grid-auto-rows The size of those rows. Important when items arrive from data.

grid-auto-flow **row** by default. **column** flips it.

dense Lets a later, smaller item climb back into a hole. Changes visual order, not the DOM — so tab order stops matching the screen.

```
1 .gallery {  
2   display: grid;  
3   grid-template-columns: repeat(3, 1fr);  
4   grid-auto-rows: 180px;  
5 }  
6  
7 .gallery {  
8   grid-auto-flow: dense;  
9 }
```

This is how a list of unknown length lays itself out.

ALIGNMENT, IN BOTH DIRECTIONS AT ONCE

THE TRACKS INSIDE THE CONTAINER

`justify-content` ·
`align-content`

Used when the tracks are narrower than the container.

THE ITEMS INSIDE THEIR CELLS

`justify-items` · `align-items`

And `justify-self` /
`align-self` for one item.

THE GOTCHA FROM LAST WEEK

In Flexbox, `justify-` follows `flex-direction`. In Grid it is **always** the inline axis. It does not turn.

► **SHORTHAND** — `place-items: center` is both.

SUBGRID, IN ONE SLIDE

A grid item that is itself a grid normally invents its own rows. With `subgrid` it takes them from the track it sits in — so the third line of every card in a row is the *same* line.

It is the honest answer to "make the footers line up when the cards hold different amounts of text".

```
1 .card {  
2   display: grid;  
3   grid-template-rows: subgrid;  
4   grid-row: span 4;  
5 }  
6  
7 @supports (grid-template-rows: subgrid) {  
8   /* wrap it, and keep the flex version  
9     as the fallback */  
10 }
```

ACT 5 · GRID OR FLEX

THE SAME CARD ROW, THREE WAYS

demos/06-three-ways — one width slider, three containers

↻ reload

open ↗

container width  760px cards  6

FLEXBOX — flex: 1 1 200px; flex-wrap: wrap

Latency

A slightly longer line of body copy.

[view](#)**Errors**

A shorter note.

[view](#)**Traffic**

A slightly longer line of body copy.

[view](#)**Saturation**

A slightly longer line of body copy.

[view](#)**Uptime**

A shorter note.

[view](#)**Cost**

A slightly longer line of body copy.

[view](#)

GRID — repeat(3, 1fr)

Latency

A slightly longer line of body copy.

[view](#)**Errors**

A shorter note.

[view](#)**Traffic**

A slightly longer line of body copy.

[view](#)**Saturation**

A slightly longer line of body copy.

[view](#)**Uptime**

A shorter note.

[view](#)**Cost**

A slightly longer line of body copy.

[view](#)► **HANDS-ON** demos/06-three-ways/ — find a width where all three agree, then break it.

THE RULE, AND IT IS SHORT

GRID

The **page**. Anything with rows and columns that have to line up.

Shells, card grids, dashboards, galleries, forms with a label column.

FLEX

The **inside of a component**. One axis, and the sizes come from the content.

Navbars, button rows, a card's internals, toolbars, a chip with an icon.

NOT A COMPETITION

You don't have to choose one over the other — Flexbox can be used inside a grid cell.

LIVE CODING — THE PAGE SHELL, FROM NOTHING

- 1 Five regions, stacked. One column, five names, no media query.
- 2 Give each region its `grid-area`. Names only, no numbers.
- 3 One breakpoint: two columns. Rewrite *only* the picture.
- 4 Second breakpoint: three columns. Same again.
- 5 Break it on purpose — make one row short a cell.

► **LIVE CODING BREAK #1** — 8 minutes. Type it, do not paste it.

ACT 6 · RESPONSIVE

NOW, THE WIDTH YOU DID NOT DESIGN FOR

Everything so far adapts **by itself**.

`fr`, `minmax` and `auto-fit` got
you a long way with no query at all.

A media query is for the decisions the content cannot make: **this is a phone, so put the nav away.**



MIN-WIDTH COMPOSES. MAX-WIDTH FIGHTS.

demos/08-responsive/01-breakpoints – same output, two sources

viewport  380px

nav

A headline that has to fit
The width at which this stops fitting is the breakpoint. Not 768.

aside

MOBILE-FIRST — MIN-WIDTH

```
.page {  
  grid-template-columns: 1fr;  
}  
@media (min-width: 560px) {  
  .page { grid-template-columns: 1fr 1fr; }  
  .nav { grid-column: 1 / -1; }  
}  
@media (min-width: 860px) {  
  .page { grid-template-columns: 200px 1fr 200px; }  
}
```

4 declarations, 1 block(s) applying now

DESKTOP-FIRST — MAX-WIDTH

```
.page {  
  grid-template-columns: 200px 1fr 200px;  
}  
@media (max-width: 859px) {  
  .page { grid-template-columns: 1fr 1fr; }  
  .nav { grid-column: 1 / -1; }  
}  
@media (max-width: 559px) {  
  .page { grid-template-columns: 1fr; }  
  .nav { grid-column: auto; }  
}
```

► **HANDS-ON** [demos/08-responsive/01-breakpoints/](#) — drag slowly and watch which blocks light up.

WHERE THE NUMBERS COME FROM

NOT THIS

```
/* phone */ 320px  
/* tablet */ 768px  
/* laptop */ 1024px
```

A table of devices from 2015. None of those phones exists any more.

THIS

Narrow the window until **your** content stops working. That width is the breakpoint, and it moved when the content changed.

Write it in `em`, so it moves with the reader's font size too.

► **SAY IT OUT LOUD** — you are designing for content, not for a catalogue of hardware.

TYPE THAT DOES NOT STEP

demos/08-responsive/02-clamp ↶ reload open ↷

viewport  480px ☒ keep the rem term in clamp() ☐ user has set a larger default font

font-size: 40px

Layout that fits the page

THREE STEPS, THREE MEDIA QUERIES

Layout that fits the page

CLAMP(24PX, 1REM + 2.5VW, 44PX)

Layout that fits the page

```
viewport      480px
root font-size 16px

clamp(24px, 1rem + 2.5vw, 44px)
  minimum 24px
  preferred 28.0px <- the PREFERRED value is winning
```

```
1 h1 {
2   font-size: clamp(1.6rem, 1rem + 2.2vw, 2.6rem);
3 }
```

IMAGES, IN THE ORDER THEY BITE

demos/09-images — press load, then the two toggles in order↻ reloadopen ↗

load the images ☐ aspect-ratio: 3 / 2 ☐ object-fit: cover

A CARD ROW, AND THREE PICTURES THAT ARE NOT THE SAME SHAPE

3 : 2

2 : 3

1 : 1

row height now 67px

Nothing is loaded yet. Note the row height – then press the button and watch it change. That change is Cumulative Layout Shift, and it is the thing that moves a button out from under somebody's finger.

```
<!-- srcset and sizes: what the browser does BEFORE it downloads anything -->

```

► **HANDS-ON** `demos/09-images/` — read the layout shift number out loud before you fix it.

srcset AND sizes, BRIEFLY

```
1 
```

`sizes` is a media condition and not a selector, because the browser has to choose a file **before** layout has happened.

POLISH, AND THE PROPERTY YOU MUST NOT ANIMATE

demos/10-motion – tick "busy" and run it again

run both

☐ make the main thread busy

ANIMATING inset-inline-start AND width — LAYOUT EVERY FRAME

left

ANIMATING transform — COMPOSITOR ONLY

transform

frames drawn0

average frame rate not measured yet

longest frame — (16.7ms is one frame at 60fps)

frames over budget 0

ANIMATE

transform · opacity · colour

DO NOT

width · height · top · left · margin — and
transition: all

THE QUERY A COMPONENT ASKS ABOUT ITSELF

A media query asks about the **window**. But a card does not care how wide the window is — it cares how wide *it* is, and the same card might sit in a sidebar and in a full-width row on the same page.

Container queries ask the second question. **Not examined, not needed for the assignment** — but it is where this is going, and it is the reason "breakpoints come from content" is more than a slogan.

```
1 .panel {  
2   container-type: inline-size;  
3 }  
4  
5 @container (min-width: 30em) {  
6   .card {  
7     grid-template-columns: 8rem 1fr;  
8   }  
9 }
```

LIVE CODING — MAKE THE CARD ROW BREATHE

- 1 Start from `repeat(3, 1fr)`. Narrow the window. It is awful.
- 2 Swap to `repeat(auto-fit, minmax(15rem, 1fr))`.
- 3 Add a long unbroken word to one card. Watch it burst.
- 4 Fix it with `minmax(0, 1fr)` on the parent track.
- 5 Add a hover with `transform`, and a reduced-motion query.

► **LIVE CODING BREAK #2** — 7 minutes.

THREE QUESTIONS

- 1 Three columns. How many lines, and what is the last one also called?
- 2 Six cards, room for four columns. What is different between `auto-fit` and `auto-fill` here?
And with three cards?
- 3 Your sidebar refuses to shrink and the page scrolls sideways. One declaration.



TONIGHT, AND THIS WEEK

IN THE LAB — CA4

A painting and a chessboard as warm-ups, then a dashboard that lays itself out three ways.

One rule: not a single `max-width` query.

HOME ASSIGNMENT 2

Rebuild a real page from a screenshot. Grid and Flex, mobile-first.

Due before week 6.

BEFORE NEXT WEEK

Nothing to install. Next week is Tailwind, and it is all of this with new names.

Optional: CSS Grid Garden, twenty minutes.